

Communication and Collaboration Plan

Phase 8 · Project Demonstration

Aishwarya Janani M (Team Lead) · Arishwaran A · Nithya Dharshni S · Dharshini M · Dharshini M

This document records how the four-member team coordinated the build of the Script-Controlled ACL project (Restrict Record Access Based on Field Value) on ServiceNow Personal Developer Instance <https://dev211458.service-now.com>. It captures member roles, the tools used, the working cadence, a RACI-style responsibility matrix, and how doubts were raised and resolved during the capstone.

Team Members and Roles

Member	Role	Primary responsibility on the build
Aishwarya Janani M	Team Lead / Instance owner	Owns the PDI, performs security_admin elevation, coordinates the build, authors the READ ACL script
Arishwaran A	Data model builder	Creates the u_institution_details table, its columns, and the Branch choice field (ECE/EEE/CSE)
Nithya Dharshni S	Security roles builder	Creates roles bb1, bb2, bb3, bb4 and the EEE User test account holding all four roles
Dharshini M	ACL configuration	Configures the four record-level ACLs (read/create/write/delete), the Advanced flag, the Branch is EEE condition, and the script
Dharshini M	Testing and documentation	Loads the four sample records, runs impersonation verification, prepares evidence and submission artifacts

Tools Used

Tool	Purpose
ServiceNow PDI (dev211458)	Single shared build and demonstration environment for the table, roles, and ACLs
Impersonation + Background Scripts	Verifying access outcomes per role without separate logins
GitHub	Version-controlled home for the READ ACL script, notes, and screenshots
SmartInternz / SkillWallet	Capstone phase tracking and final deliverable submission

Working Cadence

- One shared PDI: only the Team Lead held admin, so config changes were sequenced to avoid two people editing the same ACL at once.
- Short daily sync to confirm the next object to build (table, then roles, then ACLs, then records, then test).
- Build order was fixed and dependency-driven so each member could start only once the prior object existed.
- Every ACL change was validated by impersonation immediately after it was saved, not batched to the end.
- Screenshots of each verification (EEE User, Abel Tuter, admin) were captured and stored as they happened.

RACI Matrix

Activity	Responsible	Accountable	Consulted	Informed
Create u_institution_details table + fields	Arishwaran	Aishwarya	Dharshini	Nithya, Dharshini
Create roles bb1-bb4 + EEE User	Nithya	Aishwarya	Arishwaran	Dharshini
security_admin elevation	Aishwarya	Aishwarya	Dharshini	Team
Configure 4 ACLs (read/create/write/delete)	Dharshini	Aishwarya	Nithya	Arishwaran, Dharshini
Remove default u_institution_details_user role from ACLs	Dharshini	Aishwarya	Nithya	Team
Load sample records (ECE/EEE/CSE)	Dharshini	Arishwaran	Dharshini	Team
Impersonation verification	Dharshini	Aishwarya	Dharshini	Team
Documentation and submission	Dharshini	Aishwarya	Team	SmartInternz mentor

Doubt and Escalation Handling

- Configuration doubts (for example why an ACL did not take effect) were raised in the group chat with a screenshot and resolved in the daily sync.
- Access-control questions were settled by test, not opinion: the member impersonated EEE User or Abel Tuter and read the actual result.
- The Team Lead was the single point of decision for anything needing admin or security_admin rights.
- Blockers that could not be reproduced were escalated to the SmartInternz mentor with the exact ACL, condition, and script attached.